

Homework3 报告

本次作业中，我复现了PointNet 和 PointNet++ 实现在ModelNet40上的分类任务。

Homework3 报告

下载链接

PointNet

方法简述

代码结构

1. 数据读取

1) 基于原 OFF 文件的版本

2) 基于 sampled 点云数据的版本

2. 模型结构

3. 训练、参数和权重模型

训练曲线与结果

PointNet++

方法简述

代码结构

1. 数据加载

2. 模型结构

3. 训练、参数与权重模型

训练曲线与结果

对比分析

参数量统计

耗时与分类效果

下载链接

权重模型下载链接：

<https://disk.pku.edu.cn/link/AA0D53BC64E46B48FEA1CF60164E4E4241>

文件名：checkpoints.zip

有效期限：永久有效

数据集（resample处理之后的）下载链接：

<https://disk.pku.edu.cn/link/AAAEC6796AB9494DBFA6F84D1344F59923>

文件名：archive (1).zip

有效期限：永久有效

提取码：shA9

PointNet

方法简述

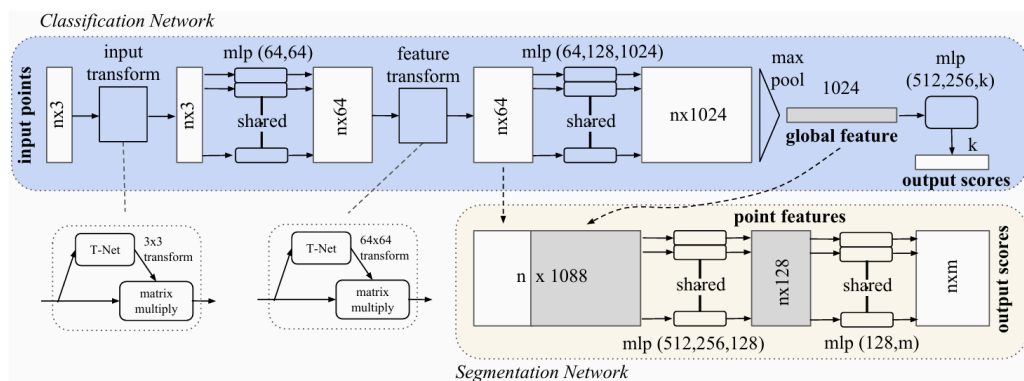
PointNet 方法的核心思想是：

- 使用共享的 MLP（Shared MLP）分别提取每个点的局部特征；
- 使用对称函数（Symmetric Function）进行全局特征聚合，从而保证模型对点输入顺序的不变性；
- 使用最大池化（Max Pooling）获得全局特征；

- 最终通过全连接层完成分类任务。

PointNet 中还引入了 T-Net 结构，用于学习输入点云以及中间特征的空间变换矩阵，从而提升模型对旋转、平移等几何变换的鲁棒性。复现 PointNet 在 ModelNet40 数据集上实现分类任务，参考原论文的结构实现，包括：

1. 输入变换网络 (Input Transform T-Net)
2. 点特征提取模块 (Shared MLP)
3. 特征变换网络 (Feature Transform T-Net)
4. 全局最大池化 (Global Max Pooling)
5. 分类器 (MLP + Dropout)



代码结构

1. 数据读取

1) 基于原 OFF 文件的版本

最初使用的是原始 ModelNet40 数据集中的 .off 网格模型文件。以这个作为原始数据训练 PointNet 需要预处理：

- 读取 OFF 文件中的顶点与三角面片
- 在三角面片表面进行随机采样
- 将网格模型转换为点云输入

在文件夹 PointNet/dataset.py 中实现，主要包括：

- `read_off()`：读取 OFF 文件
- `sample_surface()`：按照三角形面积进行表面采样
- `normalize_unit_sphere()`：归一化到单位球

虽然这个实现更贴合原论文中的数据处理流程，但由于需要在线进行 mesh surface sampling，数据预处理开销较大，实际训练过程中发现速度过慢，所以换用下面的方式，找到已经 sample 好的 modelnet40 作为初始数据。

2) 基于 sampled 点云数据的版本

使用的是 kaggle 上下载的 modelnet40_normal_resampled 数据集，其结构形如：

```
modelnet40_normal_resampled/
├── airplane/
│   ├── airplane_0001.txt
│   └── airplane_0002.txt
```

```
|   |— ...
|   |— bathtub/
|   |— bed/
|   |— bench/
|   |— bookshelf/
|   |— ...
|   |
|   |— modelnet40_shape_names.txt （类别的名称，airplane, bed,...）
|   |— modelnet40_train.txt （包含所有类别文件夹下属于train集的.txt文件名称）
|   |— modelnet40_test.txt （包含所有类别文件夹下属于test集的.txt文件名称有）
|   |— ...
```

其中，每个类别文件夹中的 `.txt` 文件对应一个已经完成点云采样的三维模型，有若干行，每一行表示一个点，共包含 6 个浮点数 x, y, z, nx, ny, nz ，分别表示 (x, y, z) -点的三维坐标， (nx, ny, nz) -该点对应的法向量

形如：

bookshelf_0002.txt:

```
0.590500,0.507100,0.104900,1.000000,0.000000,0.000000
-0.584500,-0.796900,-0.130100,-0.057650,-0.997700,-0.036080
-0.591300,0.793000,-0.122400,-0.972700,0.231300,-0.019960
0.590600,-0.699400,0.130200,0.989600,0.010220,0.143200
```

复现PointNet时，仅使用前三维 (x, y, z) 的数据即可；使用 `modelnet40_normal_resampled` 数据集不再需要再在线 mesh sampling，数据加载速度提升，有效提高了训练效率。

也可以在这个链接下载 `modelnet40_normal_resampled` 数据集：

<https://disk.pku.edu.cn/link/AAAEC6796AB9494DBFA6F84D1344F59923>

文件名：archive (1).zip

有效期限：永久有效

提取码：shA9

这个的数据加载过程在 `PointNet/dataset_new.py` 中实现。

2. 模型结构

模型定义位于 `PointNet/model.py` 中，主要包括以下几个类：

- `SharedMLP`：使用 `1x1 Conv` 实现逐点 MLP
- `TNet`：学习输入空间与特征空间变换
- `PointNetEncoder`：提取全局点云特征
- `PointNetCls`：最终分类网络

网络结构与原论文中的 PointNet Classification 基本一致：

```
64 --> 64 --> 64 --> 128 --> 1024
```

之后通过全局最大池化获得 1024 维全局特征，再经过全连接层完成 40 类分类。

3. 训练、参数和权重模型

训练代码位于 `PointNet/train.py` 中，主要包括：

损失函数包括分类交叉熵损失和Feature Transform Regularization，总体损失为 $L = L_{cls} + \lambda L_{reg}$ ，其中 $\lambda = 0.001$ 用于约束特征变换矩阵接近正交矩阵。

在RTX 4090上训练，训练好的权重模型的下载链接为：

<https://disk.pku.edu.cn/link/AA0D53BC64E46B48FEA1CF60164E4E4241>

文件名：checkpoints.zip

有效期限：永久有效

训练参数为：

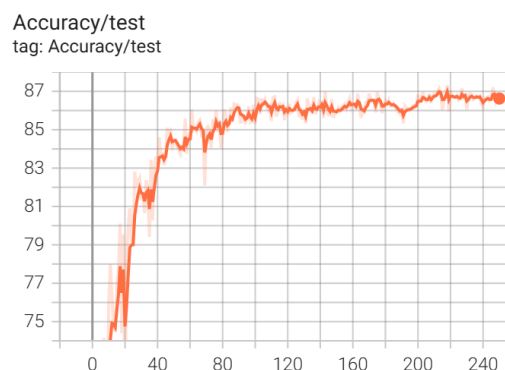
```
"num_points": 1024,  
"batch_size": 32,  
"epochs": 250,  
"lr": 0.001,  
"weight_decay": 0.0001,  
"reg_weight": 0.001,  
"step_size": 20,  
"gamma": 0.7,  
"dropout": 0.3,  
"num_workers": 4
```

也可以在上述链接解压缩后的 `checkpoints/pointnet_resampled/config.json` 中看到更多具体设置。

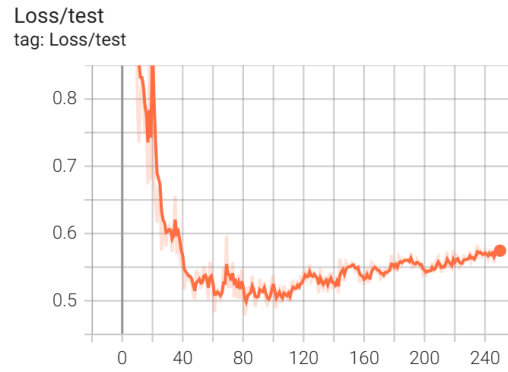
训练曲线与结果

使用 TensorBoard 记录训练过程中的 Loss 与 Accuracy 曲线，训练过程中，每个 epoch 后都会将结果写入 TensorBoard 日志文件，250轮结束后的曲线图如下（smoothing = 0.6）：

Test accuracy:

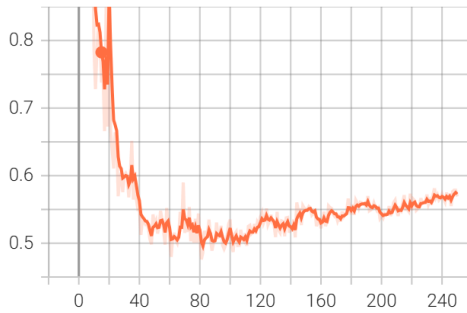


Loss:

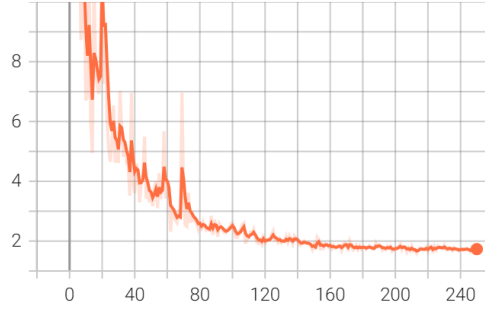


其中，Loss classification 和 Loss regularization分别如下：

Loss_classification/test
tag: Loss_classification/test



Loss_regularization/test
tag: Loss_regularization/test



(Loss classification曲线里，后面轮数中曲线有上升趋势，出现略微的过拟合现象)

从训练曲线可以观察到，模型在前期收敛较快，接近第100轮后accuracy趋近稳定；最终模型在 ModelNet40 测试集上取得了较高的分类准确率 **Best test accuracy = 87.32 %**，与原论文实验结果

	input	#views	accuracy avg. class	accuracy overall
SPH [11]	mesh	-	68.2	-
3DShapeNets [28]	volume	1	77.3	84.7
VoxNet [17]	volume	12	83.0	85.9
Subvolume [18]	volume	20	86.0	89.2
LFD [28]	image	10	75.5	-
MVCNN [23]	image	80	90.1	-
Ours baseline	point	-	72.6	77.4
Ours PointNet	point	1	86.2	89.2

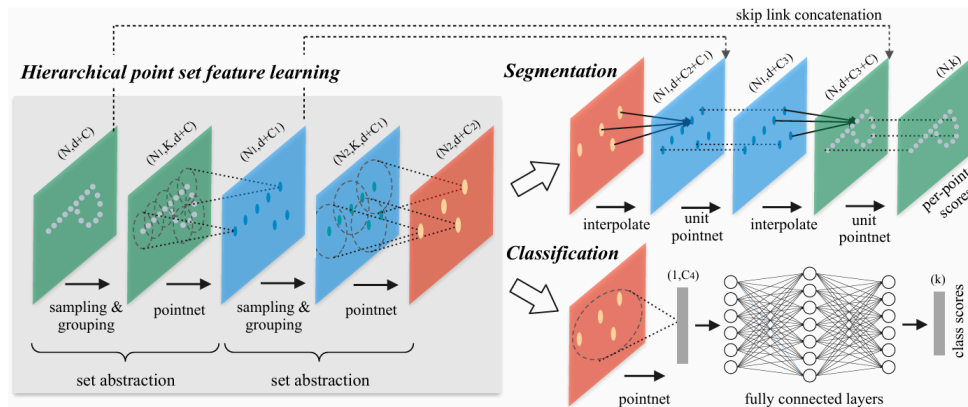
Table 1. **Classification results on ModelNet40.** Our net achieves state-of-the-art among deep nets on 3D input.

的 89.2% overall accuracy十分相近。

PointNet++

方法简述

PointNet 直接对整幅点云进行全局特征提取，虽然能够处理无序点集，但对局部几何结构的建模能力较弱；因此，PointNet++ 的核心思想是借鉴 CNN 中的层次化特征提取方式，在点云上逐层构建局部区域，并在每个局部区域内使用 PointNet 提取局部特征，从而学习由局部到全局的几何表示。



其基本模块是 Set Abstraction (SA) 模块，每个 SA 模块主要由三部分组成：Sampling Layer、Grouping Layer 和 PointNet Layer。Sampling Layer 通常使用最远点采样 (Farthest Point Sampling, FPS) 选取局部中心点；Grouping Layer 根据中心点在一定半径内搜索邻域点；PointNet Layer 对每个局部邻域中的点进行共享 MLP 和最大池化，得到该局部区域的特征表示。通过多层 SA 模块，网络逐步降低点的数量，同时提升特征维度，最终得到全局点云特征。

原论文也提出了 MSG 和 MRG 等多尺度策略，用于缓解点云密度不均匀的问题。本次作业中我复现的是 PointNet++ 的 SSG (Single-Scale Grouping) 分类版本，即每个 SA 层只使用一个固定半径进行局部区域划分。

代码结构

1. 数据加载

与复现 PointNet 同理，本实验使用已经采样好的 `modelnet40_normal_resampled` 数据集。数据集中每个样本包含点的坐标和法向量信息，本实验的 PointNet++ SSG 分类模型只使用前三维 xyz 坐标作为输入。

数据加载部分主要在 `dataset_ssg.py` 中实现。代码读取 ModelNet40 的类别文件和 train/test 划分文件，并从对应的 `.txt` 点云文件中读取点坐标。每个样本被随机采样为固定数量的点，默认使用 1024 个点。训练阶段加入了归一化、绕 y 轴随机旋转和 jitter 噪声增强；测试阶段只进行固定采样和归一化。

2. 模型结构

模型结构主要在 `train_ssg.py` 中实现。本实验复现的是 PointNet++ SSG 分类网络，整体结构为：

```
SA1 -> SA2 -> SA3(global) -> FC classifier
```

其中三个 SA 模块具体为：

SA1: npoint=512, radius=0.2, nsample=32, MLP=[64, 64, 128]

SA2: npoint=128, radius=0.4, nsample=64, MLP=[128, 128, 256]

SA3: global abstraction, MLP=[256, 512, 1024]

在每个非全局 SA 层中，代码先通过 FPS 采样中心点，再通过 ball query 查找局部邻域，再用共享 MLP 和 max pooling 提取局部特征；最后一层 SA 将点云特征聚合为全局特征，随后经过三层全连接分类头输出 40 类分类结果：

```
1024 -> 512 -> 256 -> 40
```

3. 训练、参数与权重模型

训练部分在 `PointNet++/train_ssg.py` 中完成。损失函数使用交叉熵损失，优化器使用 Adam，默认训练参数包括：

```
"num_points": 1024,  
"batch_size": 16,  
"epochs": 250,  
"lr": 0.001,  
"weight_decay": 0.0,  
"decay_step": 200000,  
"decay_rate": 0.7,  
"min_lr": 1e-05,  
"bn_init_momentum": 0.5,  
"bn_decay_rate": 0.5,  
"bn_min_momentum": 0.01,  
"dropout": 0.5,  
"num_workers": 4
```

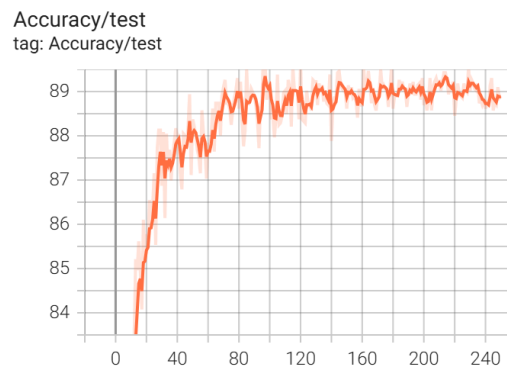
也可以在权重文件下载链接解压后的 `checkpoints/pointnet2_ssg/config.json` 中看到更多具体设置。

模型权重保存在 `checkpoints/pointnet2_ssg/` 目录下，包含训练过程中测试准确率最高的模型 `best_model.pth`

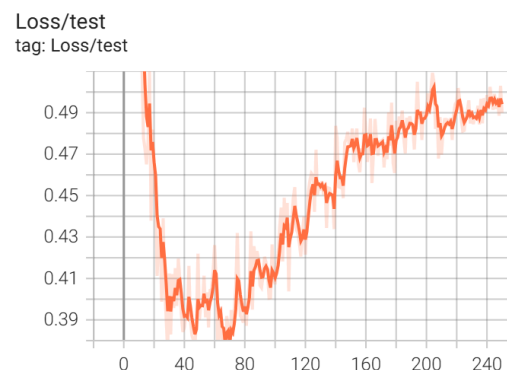
训练曲线与结果

同样使用 TensorBoard 记录训练过程中的 Loss 与 Accuracy 曲线，250轮结束后的曲线图如下（smoothing = 0.6）：

Test accuracy:



Loss:



(同样在loss曲线里，后面轮数中有上升趋势，出现比PointNet更明显的过拟合现象)

从训练曲线可以观察到，模型在前期收敛较快，接近第100轮后accuracy趋近稳定；最终模型在 ModelNet40 测试集

上取得了较高的分类准确率 **Best test accuracy = 90.03 %**，与原论文实验结果

Method	Input	Accuracy (%)
Subvolume [21]	vox	89.2
MVCNN [26]	img	90.1
PointNet (vanilla) [20]	pc	87.2
PointNet [20]	pc	89.2
Ours	pc	90.7
Ours (with normal)	pc	91.9

Table 2: ModelNet40 shape classification.

Ours = 90.7% accuracy十分相近（即对标论文中 xyz-only 设置，暂没复现比对未 MSG/MRG 或 with-normal 的实验）

对比分析

参数量统计

训练代码 PointNet/train.py 和 PointNet++/train_ssg.py 中均通过如下函数统计模型参数量：

```
def count_parameters(model):
    return sum(parameter.numel()
                for parameter in model.parameters()
                if parameter.requires_grad) #只统计可训练参数 requires_grad=True
```

最终在训练开始前输出模型总参数量：`print(f"Trainable parameters:{count_parameters(model):,}")`，具体输出为：

```
PointNet
Trainable parameters: 3,473,969

PointNet++
Trainable parameters: 1,472,360
```

发现 PointNet++ 参数量反而小于PointNet，可能因为：PointNet 包含输入变换网络和特征变换网络 T-Net，尤其是特征变换网络需要预测一个 64*64 的变换矩阵，其最后输出维度为 4096，因此会引入较多额外参数；而PointNet++ SSG通过 Set Abstraction 模块中的共享 MLP 对局部邻域进行特征提取，MLP在不同点和局部区域之间复用参数，不引入过多可训练参数，参数量更小。

耗时与分类效果

运行 PointNet++/train_ssg.py 训练PointNet++总共耗时7h 49min，平均约2min训练一个epoch。运行 PointNet/train.py 训练PointNet平均约 80s训练一个epoch，耗时短于PointNet++

--> PointNet 的结构相对简单，主要对所有输入点共享 MLP，并通过一次全局 max pooling 得到整体特征；相比之下，PointNet++ 在每一层 Set Abstraction 中都需要进行最远点采样（FPS）、ball query 邻域搜索和局部特征聚合，这些操作会引入额外的计算开销。同时作业在复现时，FPS 和 ball query 使用 PyTorch 实现 (而非高度优化的 CUDA 扩展)，可能进一步增加训练耗时。

从test accuracy看，PointNet++ (acc=90.03%) 分类效果优于PointNet (acc=87.32%)

--> PointNet 直接从整幅点云中提取全局特征，对点云的局部邻域关系利用较弱；而 PointNet++ 通过层次化的 SA 模块逐步从局部区域到全局形状提取特征，使网络能够捕捉更细粒度的信息如局部曲面、部件结构和空间组合关系，因此在 ModelNet40 分类任务中它取得更好的分类效果。